Welcome to your next lesson: Functions iii: return vs print and early return.

I'm preparing your personalized lesson based on what you've learned, what's coming up, and the best way to help you move forward.

Step by step, you're moving forward.

- Imagine a function that calculates the total cost of an order ?

Sometimes we want:

- to show the total to the screen (for a human) → `print()`
- to use that total in more math (for the program) → `return`

Very simple picture:

```
Text
print()  → sends text to the screen, then it's gone
return   → sends a value back to the code, so it can be reused
```

We'll go step by step and play with both.

Quick check-in:

When you write a function, what do you feel less confident about right now?

A) Knowing when to use `print()` vs `return`

B) Using a returned value later in the code

C) Early `return` (ending a function early)

D) Honestly, all of the above ?

> fairly cnfident an all which isnt a chice but ill go with C as we havent done conditiona exits

Nice, that helps a lot — and good point: we haven't really played with early exits yet, so C makes sense ?

Let's focus on one idea first:

A function can stop in the middle using `return`. Everything after that `return` is skipped.

Very simple example:

```python
def demo():
    print("start")
    return 42
    print("you will never see this")

result = demo()
print(result)
```

Step-by-step:

1. `demo()` starts.

2. It prints `"start"`.
3. It hits `return 42` → function ends right there.
4. The last `print("you will never see this")` is never run.
5. Outside, `result` becomes `42`, then we print it.

If you ran this, you'd see:

```
start
42
```

No third line.

Does that flow match what you expect, or is any step in there even a little fuzzy?

That makes sense

Great ?

Now I want to zoom in on why we'd end a function early — this is the idea behind a guard clause.

Think of it like this:

You're writing a function to calculate a discount, but if the price is `0` or negative, you just want to bail out early and return `0` right away — no more work, no more math.

Very rough shape (don't worry about the exact condition yet):

```python
def calculate_discount(price):
    # if price is not valid → return early
    # (guard clause lives here)

    # normal work happens down here
    discount = price * 0.1
    return discount
```

So a guard clause is just:

a quick check at the top of a function that can `return` early and skip the rest.

Before we code anything:

Does this idea of "quick checks at the top that can bail out early" feel natural to you from other languages / experience, or is this pretty new?

I've seen it in other languages

Perfect, then we can lean on that and just make sure it's clear in Python terms ?

One key detail in Python:

`return` does two things at once:

1. It stops the function.

2. It hands back a value (or None if you don't give one).

So an early return / guard clause is just using that stopping power a bit sooner.

Example:

Python

```
def safe_divide(a, b):
    if b == 0:
        return None  # bail out early

    result = a / b
    return result
```

Flow:

- If b == 0 → function ends immediately, returns None.
- Otherwise → it skips the if, does the division, and returns the result.

Want to try a tiny one like this together next, or would you rather first look at a super simple return vs print contrast to lock that in?

sure we can try one you just snuck a new operator in on me though == because a = b woldnt compare the it would assign b content to a but == compare

Nice catch — and you're exactly right ?

In Python:

- a = b → assignment (put b's value into a)
- a == b → comparison (check if they have the same value, gives True or False)

So in a guard clause like:

Python

```
if b == 0:
    return None
```

I'm asking, "Is b equal to zero?" — not assigning.

Let's try a tiny guard-clause function together.

Goal: if b is 0, return early. Otherwise, do the division.

In the editor, write a function named safe_divide that:

1. Takes two parameters: a and b.
2. If b is 0, returns None immediately (early return).
3. Otherwise, returns a / b.

Then, under the function, add:

Python

```
print(safe_divide(10, 2))
print(safe_divide(5, 0))
```

Run it, then hit Submit code, and I'll walk through the behavior with you.

```python
def safe_divide(a, b):
    if b == 0:
        return None  # bail out early

    result = a / b
    return result
print(safe_divide(10, 2))
print(safe_divide(5, 0))
```

Nice work, Ron — that's exactly the pattern I was going for ✅

Let's read your code step by step:

```python
def safe_divide(a, b):
    if b == 0:
        return None  # bail out early

    result = a / b
    return result

print(safe_divide(10, 2))  # → 5.0
print(safe_divide(5, 0))   # → None
```

- `safe_divide(10, 2)` → `b` is not 0, so it skips the early `return`, does `10 / 2`, and returns `5.0`.
- `safe_divide(5, 0)` → `b == 0` is `True`, so it returns `None` immediately and never runs the division line.

This is a clean example of an early return / guard clause. ✅

Next, I want to connect this back to `print` vs `return` in a very concrete way.

Imagine this version:

```python
def safe_divide_print(a, b):
    if b == 0:
        print("None")
        return

    result = a / b
    print(result)
```

Here we only print inside the function; we don't give the caller any value to reuse.

Question for you (no need to code, just your reasoning):

If I write this:

```
Python
result = safe_divide_print(10, 2)
```

what will be stored in **result**?

> no result is passed back so whatever the default of result is y guess would be 0 as the def of he function seems to imply ending ints

Love how you're thinking through that ?

In Python, if a function finishes without an explicit **return** some_value, it actually returns None, not 0.

So:

```
Python
def safe_divide_print(a, b):
    if b == 0:
        print("None")
        return   # this is the same as: return None

    result = a / b
    print(result)   # no return here
```

If you do:

```
Python
result = safe_divide_print(10, 2)
```

- It will print 5.0 to the screen.
- But the function itself doesn't **return** a value, so Python secretly does **return None**.
- So **result** will actually hold None, not 0.

Same for **safe_divide_print**(5, 0): it prints "None", then returns None.

Key idea to lock in:

> ***print***() → *shows text on screen, but the function still returns None unless you explicitly* ***return*** *something.*
>
> ***return*** *value* → *gives that value back to the caller so it can be stored and reused.*

Want to try a quick tiny exercise around this?

I'll ask you to predict what will be printed vs what a variable will store.

> Yes and actually i intersted in that 2 variables are ent in ye weve only discussed sending 1 or none back, im curious if the function were meant to add +1 to each variablkle et in , and return the result both of them to the caller

Nice question — that's exactly the kind of thing good devs wonder

about ?

Step by step:

1. A function can take multiple parameters (like your a and b).
2. A function can only do one return value at a time.
3. But that one value can itself hold multiple things (we'll do that properly later with tuples/lists).

For now, stay inside today's lane: think of it like this:

```python
def add_one_to_each(a, b):
    new_a = a + 1
    new_b = b + 1
    # somehow send both new_a and new_b back
```

In this lesson, we're focused on:

- how return stops the function
- how it sends a value back
- how you then capture that value and reuse it

We'll get to "returning multiple things in one shot" a bit later in the course.

For now, want to: A) Keep going with single return values (return vs print)

or

B) Do a tiny preview of how Python does let you return two values at once, just as a sneak peek?

gotvcha

Nice ?

Let's do that quick exercise on what's printed vs what's actually returned.

Look at this code mentally:

```python
def mystery(x):
    print("Inside:", x * 2)
    return x * 3

result = mystery(4)
print("Result is:", result)
```

Step-by-step questions (just answer in words/numbers, no need to run it):

1. What does the first print (inside the function) show on the screen?
2. What does the second print (after the call) show?
3. What value is actually stored in result?

Answer those, and then we'll tie it back to early returns and guard clauses.

a is fine

Got it ?

Let's build one more small guard-clause example that also reinforces **return** vs print.

Step-by-step idea:

We'll write a function apply_coupon(price, coupon_amount) that:

1. If price is less than or equal to 0, returns 0 immediately (early return / guard clause).
2. Otherwise, subtracts the coupon from the price.
3. Returns the final price (does not print inside).

Then we'll call it a couple of times and print outside the function.

In the editor, do this:

1. Define apply_coupon(price, coupon_amount) with that behavior.
2. Under it, add:

Python
```
final1 = apply_coupon(100, 15)
final2 = apply_coupon(0, 20)

print(final1)
print(final2)
```

Run it, then hit Submit code, and I'll walk through exactly what happens line by line with you.

your eample inside prints Inside: 8 then at the end prints Result: is 12

Exactly right on both prints ?

For:

Python
```
def mystery(x):
    print("Inside:", x * 2)
    return x * 3

result = mystery(4)
print("Result is:", result)
```

You nailed it:

- Inside: Inside: 8 (because 4 * 2)
- Outside: **Result is**: 12 (because 4 * 3 was returned and stored in **result**)

So here's the clean separation:

- **print**("Inside:", x * 2) → just shows 8 on the screen, nothing is stored from that.
- **return** x * 3 → gives 12 back to the caller, which gets stored in **result**.

Want to try that apply_coupon guard-clause function next, or would you rather do one more quick mental trace like this?

question you didnt speify how to chek if price <= 0 that syntax does not work

Great catch again ? — you're right to question the syntax.

The idea I described was:

> *"If price is less than or equal to 0, return 0"*

In Python, the comparison operator for "less than or equal" is:

```python
price <= 0
```

But that has to live inside something (like an `if`) to actually do anything, e.g.:

```python
def apply_coupon(price, coupon_amount):
    if price <= 0:
        return 0      # early return / guard clause

    final_price = price - coupon_amount
    return final_price
```

So the pieces are:

- `<=` → "less than or equal to" (comparison operator, like `==`, `<`, `>`).
- `if price <= 0:` → the actual conditional line.

You're totally right that writing just `price <= 0` on its own in a function body would not be valid in this context.

Can you walk me through, in your own words, what this `apply_coupon` would return for:

- `apply_coupon(100, 15)`
- `apply_coupon(0, 20)`

Just the two numbers is fine.

i found my error it wast the <= it was failing to notie if statement need an : as do def lines

Yep, exactly — nice self-debug there ?

In Python both:

```python
def my_function():
    ...

if something_happens():
    ...
```

need that trailing `:`. Without it, Python gets grumpy.

So your guard clause line should look like:

```python
```

```python
def apply_coupon(price, coupon_amount):
    if price <= 0:
        return 0

    final_price = price - coupon_amount
    return final_price
```

Want to try coding that `apply_coupon` version now in the editor, with a couple of test calls like:

```python
print(apply_coupon(100, 15))
print(apply_coupon(0, 20))
```

and then Submit so we can walk through what's happening?

```python
def apply_coupon(price, coupon_amount):
    if price <= 0:
        return
    price = price - coupon_amount
    return price
final1 = apply_coupon(100, 15)
final2 = apply_coupon(0, 20)

print(final1)
print(final2)
```

Nice, Ron — this is really close, and you just hit another important `return` detail ?

Your code:

```python
def apply_coupon(price, coupon_amount):
    if price <= 0:
        return
    price = price - coupon_amount
    return price

final1 = apply_coupon(100, 15)
final2 = apply_coupon(0, 20)

print(final1)
print(final2)
```

Output:

```text
```

```
85
None
```

Step by step:

1. `apply_coupon(100, 15)`
   - `price <= 0` → **False**, so it skips the early **return**.
   - `price = 100 - 15` → `85`
   - **return** price → returns `85`

   → **final1** becomes `85`, and **print**(final1) shows `85`.
2. `apply_coupon(0, 20)`
   - `price <= 0` → **True**, so it hits **return** immediately.
   - That **return** with nothing after it is the same as **return None**.

   → **final2** becomes `None`, and **print**(final2) shows `None`.

So your guard clause is working: it bails out early when `price <= 0`. ✅

If you want it to explicitly say "I am returning 0 in that case", you'd tweak just one line:

Python

```
if price <= 0:
    return 0
```

Everything else can stay the same.

Want to try that tiny change (returning `0` instead of `None`) and re-run, just to see the difference in output?

Python

```
def apply_coupon(price, coupon_amount):
    if price <= 0:
        return 0
    price = price - coupon_amount
    return price
final1 = apply_coupon(100, 15)
final2 = apply_coupon(0, 20)

print(final1)
print(final2)
```

Beautiful, Ron — that's exactly it ✨

Your code now:

Python

```
def apply_coupon(price, coupon_amount):
    if price <= 0:
        return 0        # early return with explicit value
    price = price - coupon_amount
```

```
    return price

final1 = apply_coupon(100, 15)
final2 = apply_coupon(0, 20)

print(final1)  # 85
print(final2)  # 0
```

Behavior:

- For `apply_coupon(100, 15)` → guard clause is skipped, returns `85`.
- For `apply_coupon(0, 20)` → guard clause triggers, returns `0` immediately, skipping the subtraction.

You've just:

- Used `return` to pass results out.
- Used an early return (guard clause) to simplify logic.
- Seen clearly how `return` differs from just `print`.

Before we start thinking about wrapping this lesson:

Is there anything about `print` vs `return` or early returns/guard clauses that still feels even slightly fuzzy, or does this all feel solid right now?

im pretty clear on it i believe

What do you think? ?

In Python, what is the MAIN difference between `print()` and `return` inside a function?

A.

`print()` shows a value on the screen, `return` sends a value back to the caller to be used in the code.

B.

They are the same; `print()` and `return` both give a value back to the caller.

C.

`print()` stops the function, but `return` keeps it running.

D.

`return` shows a value on the screen, but `print()` hides it from the user.

You nailed that answer, Ron ?

Your choice:

> *`print()` shows a value on the screen, `return` sends a value back to the caller to be used in the code.*

is exactly right.

Quick recap in your words:

- `print()` → for humans. It just displays text/output in the console. The value is not passed back

to the caller; it's gone as far as the program is concerned.

- **return** → for the program. It hands a value back to whatever called the function, so you can do things like:

```
result = my_function()
print(result)
another = result + 10
```

You also showed solid understanding of early returns / guard clauses with `safe_divide` and `apply_coupon` — that's exactly what this lesson was aiming for ✅

You're ready for the next step in functions.

When you're ready, hit Next lesson in the Learn tab, and I'll be there to keep going with you ?

Thank you for your feedback!

Maestro can make mistakes. Check impor